EXECUTIVE REFERENCE DECK · V0.7.0

SAP Hybris to Salesforce. Migrated by AI. Proven to run.

A complete walkthrough — the problem, the architecture, how the AI agent team works, how every output is verified, and what it means for the business. One deck, every answer.

SAP HYBRIS · JAVA → AI AGENT TEAM → SALESFORCE · APEX

VS Code extension + CLI

Powered by Anthropic Claude

63 automated tests · all passing

AGENDA

What we'll cover.

01	The problem	why manual migration fails
02	What the platform does	in → out
03	How it works	the four moves
04	Architecture	the layers, at a glance
05	The pipeline	10 stages, end to end
06	The AI agent team	who does what
07	The user journey	right-click → project
08	Self-healing verification	why you can trust it
09	Grounding & safety	no hallucinations
10	Evidence, status, security	real runs · shipped scope
11	Business case & FAQ	value, cost, questions

Replatforming by hand is slow, expensive — and quietly loses business logic.

Months → Years

A mid-size Hybris estate holds hundreds of Java classes, a custom data model, live data, and scheduled jobs. Manual rewrites are measured in quarters.

Two-platform experts

It needs engineers senior in *both* Hybris and Salesforce — among the rarest, costliest skill combinations in the market.

Silent logic loss

Rules like *"never accept a zero-value order"* vanish in manual translation. Nobody notices — until production.

IN PLAIN WORDS

Rewriting an old system by hand is like retyping a 1,000-page contract from memory — slow, and the fine print is what gets lost.

IN — a Hybris codebase

- Java business logic (orders, customers, pricing…)
- The data model (items.xml)
- Actual data records (ImpEx files)
- Scheduled jobs (cronjobs + triggers)

MINUTES–HOURS

OUT — deployable Salesforce

- Apex code in Salesforce enterprise patterns
- A test class for every class

IN PLAIN WORDS

Old system in, new system out — with a quality report on top, like a surveyor's certificate handed over with the keys.

Four moves — the way a strong engineering team would do it.

1

Understand

Reads every file, maps dependencies, and writes down the business rules it finds — before writing any code.

2

Plan

Decides what each piece becomes — including what should *not* be custom code at all.

3

Build & Review

Writes the code and tests; a second AI reviews every piece skeptically before it's accepted.

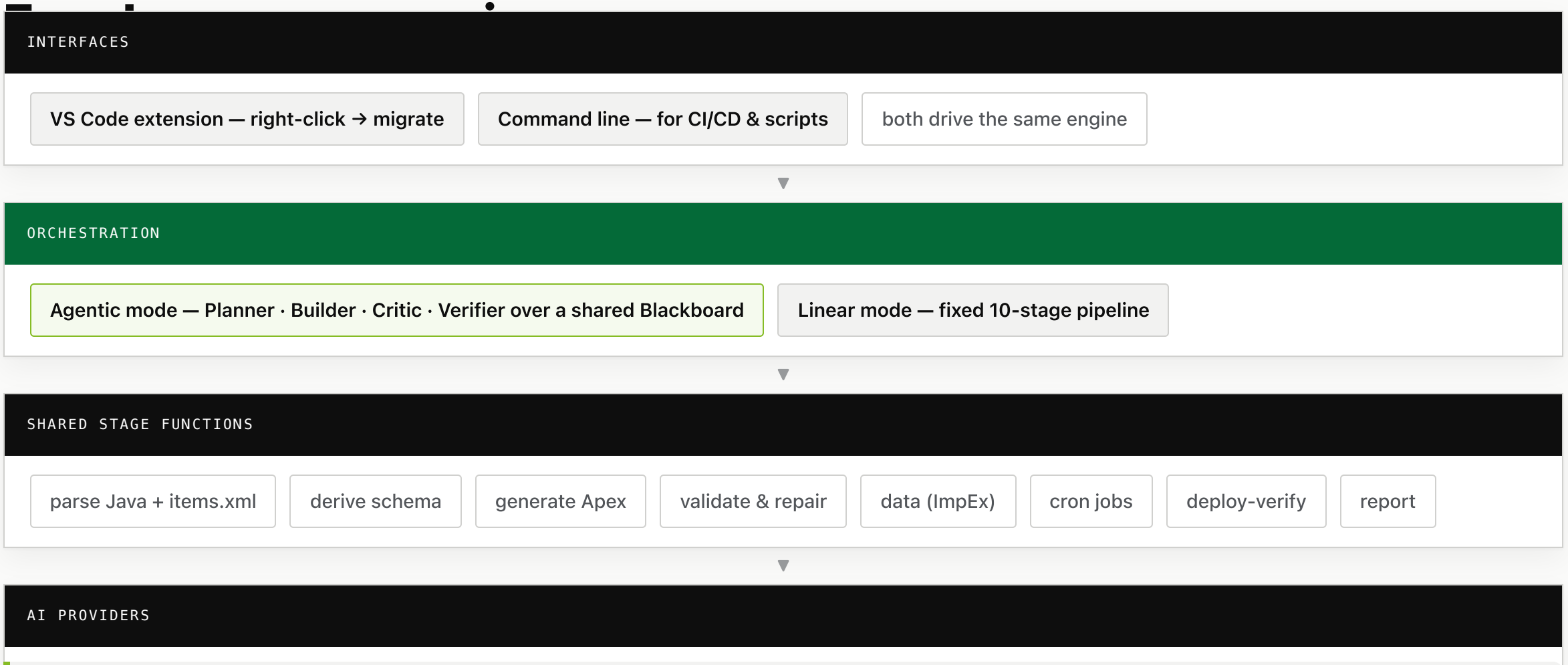
4

Prove

Deploys to a real Salesforce environment and fixes real errors itself — in a loop, until green.

IN PLAIN WORDS

Read first, decide second, build with a reviewer looking over your shoulder, and don't call it done until it actually runs. Every decision is written down — no black box.



IN PLAIN WORDS

Two "driving modes" — a smart agent team, or a simple assembly line — share the same proven tools underneath, so every fix lands in both. And the AI brain is swappable, including a free offline one.

01

Crawl & schedule

Group classes by business domain; sort so dependencies convert first.

02

Call graph

Map who-calls-whom for the visual dashboard.

03

Ingest

Parse the Java and the data-model definitions precisely (no AI needed).

04

Derive schema

Build the target object/field catalog — the single source of truth.

05 AI

Comprehend

AI summarises each class: purpose, queries, business rules.

06 AI

Generate

AI writes the Apex + tests, grounded in the schema and its dependencies.

07 AI

Validate & repair

Static safety checks; failures loop back to the AI for a targeted fix.

08

Reconcile & metadata

Fill schema gaps with evidence; emit deployable objects, fields, picklists.

09

Data & jobs

ImpEx → load-ready CSVs; cron triggers → a scheduling runbook.

10 AI

Verify & report

Real-org deploy + self-heal; confidence scores; full reports.

g

repeatable.

A manager, a shared whiteboard, and four specialists.

Orchestrator

ROUTES WORK · LOOPS UNTIL EVERYTHING IS VERIFIED

▲ ▼

The **Blackboard** — shared state: schema · migration plan · generated artifacts · **decision log** · open questions for humans

▲ ▼

STRATEGY **Planner**	EXECUTION **Builder**
QUALITY GATE **Critic**	PROOF **Verifier**

IN PLAIN WORDS

Four specialists around a shared whiteboard — so work can be sent *back* (Critic finds a problem → Builder fixes it), exactly like a real team. The whiteboard becomes a readable document after every run.

Who does what — and what each one replaces.

STRATEGY

The Planner

Routes every piece: build as **Apex**, recommend a **native Salesforce product**, or **skip** — with a written rationale.

> *"Pricing rules? That's Salesforce CPQ — don't hand-build it."*

REPLACES — blind, hard-coded translation of everything.

EXECUTION

The Builder

Writes the Apex + a test class per class, grounded in the real schema and a built-in best-practice knowledge base.

> *"Bulk-safe, secure, in the house pattern — like a senior dev."*

REPLACES — months of manual line-by-line rewriting.

QUALITY GATE

The Critic

Adversarially reviews every artifact: original **behavior** preserved? **Secure**? Real problems trigger a bounded fix-and-re-review.

> *"The zero-total rule got dropped on one path — blocked."*

REPLACES — hoping a human catches everything later.

PROOF

The Verifier

Deploys to a real org, reads **real compiler errors**, heals them, and re-deploys until green.

> *"Not 'looks right' — it actually ran."*

REPLACES — "trust me, it compiles."

From right-click to finished project.

1

Configure once

Pick the AI provider and paste your key in Settings — or choose free mock mode.

2

Right-click

On the Hybris folder in VS Code → **"H2A: Migrate to Apex."** First run sets itself up.

3

Watch

A dashboard streams progress: domains found, plan decisions, review results.

4

Receive

A **salesforce_<project>** folder appears — a complete, deployable project.

5

Review & ship

Open the feasibility report; review what's flagged; deploy with one command.

BEHIND THE SCENES the extension bundles the entire Python engine — no manual installation; the same run is scriptable from a terminal for CI/CD pipelines.

IN PLAIN WORDS

For the person using it, the whole platform is one right-click and one folder of results.

It doesn't stop at "the AI wrote code." It proves the code runs.

Deploy for real

Validation-only deploy to an actual Salesforce org

Read real errors

The actual compiler output — not a guess

Fix automatically

Three healing modes (below)

Repeat until green

Anything unresolved is flagged for a human

↺ BOUNDED LOOP — RUNS UNTIL GREEN, NEVER SILENTLY SHIPS

Metadata healing

"Missing field" error? If the field is genuinely used in the original Java — proven by evidence — it's added to the data model. Never guessed.

Code repair

Compile errors are fed back to the AI *with the real error message*, and the class is rewritten and re-tried.

Coverage healing

Salesforce requires 75% test coverage to deploy. Below it? The tool writes additional tests — error paths, bulk scenarios — until it clears the bar.

Three mechanisms that stop the AI from making things up.

1 · Schema grounding

Before writing a single query, the AI is shown the **exact catalog** of objects and fields derived from your real data model — and everything it writes is checked against that catalog afterwards. Anything that doesn't exist is caught.

2 · Evidence-based reconciliation

Generated code references a field the model never declared? The system checks the **original Java source**. Genuinely used there → added, properly typed. No evidence → flagged for human review. **Never silently guessed.**

3 · A built-in knowledge base

Salesforce's governor limits, security rules, and enterprise patterns are bundled as a reference library the agents **retrieve and cite** while writing and reviewing — facts, not memory.

IN PLAIN WORDS

The AI works "open book": it can only use what provably exists, its claims are checked against the source, and it looks rules up instead of recalling them.

What lands in the output folder.

`force-app/…/classes` **Apex code + tests**

Selectors, Services, REST controllers, scheduled jobs — each with its own test class.

`force-app/…/objects` **Data model**

Custom objects, fields, picklists, relationships, required/unique constraints.

`data/ + DATA_MIGRATION.md` **The data**

Load-ready CSVs plus a safe, re-runnable import runbook.

`CRON_JOBS.md + schedule.apex` **Schedules**

Every timed job translated, same timing, with a ready-to-run scheduling script.

`MIGRATION_PLAN.md` **The decisions**

The Planner's call on every class, the Critic's findings, and the full decision log.

`FEASIBILITY_REPORT.md` **The scorecard**

Validation results, a High/Medium/Low confidence score per class, deploy status, and cost.

IN PLAIN WORDS

Not just code — a complete package: the system, its data, its schedules, and the paper trail that tells your reviewers exactly where to look.

Two unscripted moments from a real run.

THE PLANNER'S JUDGEMENT

It refused to write unnecessary code.

Handed a custom discount/promotions engine, the Planner didn't translate it — it recommended **Salesforce CPQ**, the native product built for pricing rules, and generated no custom code for it.

> Its own words: **"Discount/promo-code pricing rules are a textbook fit for Salesforce CPQ."** Architect-level judgement — less code to own, less debt.

THE CRITIC'S CATCH

It caught a silently-lost business rule.

The original Java rejected non-positive order totals. The first translation *compiled perfectly* — but dropped that check on one path. The Critic caught it and blocked the class as "needs review."

> This is exactly the bug class that slips through manual migrations and surfaces **in production**. Here it never reached a human unflagged.

Shipped and working today.

CAPABILITY	WHAT IT COVERS	STATUS
Code migration	Java business logic → best-practice Apex, one test class per class	SHIPPED
Data model	Objects, fields, picklists, relationships, constraints — from items.xml	SHIPPED
Data records	ImpEx → load-ready CSVs + re-runnable import runbook	SHIPPED
Scheduled jobs	Cronjobs → Salesforce scheduled Apex, identical timing	SHIPPED
Agent team + self-healing verification	Planner · Builder · Critic · Verifier; deploy loop; confidence scoring	SHIPPED
Workflows & storefront APIs	Hybris processes → Flow; OCC REST → Apex REST	ROADMAP

63 automated tests, all passing

3 AI providers incl. a free offline mode

every run reports its own **cost**

Your code, your keys, your choice of where it goes.

1. **You control the AI provider**
   Source code goes only to the vendor you configure. Keys live in your settings — never in source control, never in the shipped product (audited every release).

2. **A zero-exposure mode**
   Mock mode runs the entire pipeline with **nothing leaving the machine** — for sensitive codebases and free evaluation.

3. **Secure code by default**
   Generated Apex enforces Salesforce field-level security and record-sharing as a house standard, not an afterthought.

4. **Nothing destructive, ever**
   Deployments are validation-only dry runs. A human holds the final go-live button. Every AI decision is logged and auditable.

ON THE ROADMAP

Private/VPC model hosting and approval-gated review workflows for regulated environments.

What changes, concretely.

WITHOUT THE PLATFORM	WITH THE PLATFORM
Months of manual rewriting	A working first draft in minutes–hours
Business rules silently lost	An AI reviewer checks rule preservation; a parity score proves it
"Trust me, it compiles"	Deployed to a real org and self-corrected until verifiably green
Everything becomes custom code to own forever	Native Salesforce products recommended where they fit — less debt
A black-box engagement	A decision log and a confidence score on every class

THE HONEST POSITIONING

A strong, **verified** first draft plus a prioritized review list. It makes expert reviewers dramatically faster — it doesn't remove them, and we don't pretend it does.

The questions every room asks.

Q. Does this replace our developers?

No — it replaces the grind. Experts review a scored draft instead of rewriting from scratch; confidence scores show exactly where to look.

Q. What if the AI makes something up?

Three nets: it can only use fields that provably exist; a second AI reviews every class; and the code must actually compile against a real org.

Q. Is our source code safe?

It goes only to the provider you configure — or nowhere at all in mock mode. No secrets are ever bundled; deploys are dry-run only.

Q. What does a run cost?

Every report itemizes its own AI usage. Caching keeps repeat costs low; cheaper models handle the simple steps automatically.

Q. What can't it do yet?

Hybris workflow processes and storefront REST APIs are next on the roadmap — and it tells you honestly what it skipped, and why.

Q. Can we try it on our codebase?

Yes — today. Free mock mode first (zero exposure), then a real scored run. It's a right-click in VS Code.

Built in phases — each one de-risks the next.

PHASE 0 ✓

Prove correctness

Self-healing deploys, confidence scoring, behavior-parity checks

PHASE 1 ✓

The agent team

Planner, Builder, Critic, Verifier over a shared whiteboard

PHASE 2 ◐

The whole platform

Data ✓ · schema ✓ · jobs ✓ · workflows & storefront APIs next

PHASE 3

Enterprise-grade

Review workspace, audit trail, private/VPC models, org-aware reuse

PHASE 4

It learns

Every migration makes the next better and cheaper; assessment product

THE THROUGH-LINE

Every phase moves along one axis: from output you have to *trust* → to output you can **verify**. That axis is the moat.

THE ASK

Give us one real slice of a Hybris codebase. We'll hand back verified Salesforce — and the receipts

A pilot costs days, not months. Free mock mode first, a scored real run second, your team's verdict third.

Live demo **today** | Full docs: **PRD · TDD · flows · security** | **Pilot-ready**